

EMBER

Guía de estudio, explicada bien despacito
memoria episódica bioinspirada para robots cognitivos

Notas de preparación — BIP 2026

September 8, 2026

Contents

1	La idea, contada como un cuento	2
2	El problema, un poco más en serio	2
2.1	LOLA: el desafío de fondo	2
2.2	e-MDB: la casa donde vive este problema	3
2.3	¿Por qué no alcanza con copiar lo que ya se conoce?	3
3	Los conceptos, uno por uno	3
3.1	Sparse Distributed Memory (SDM): guardar un secreto repartido	3
3.2	El engrama: la huella que deja un recuerdo	4
3.3	Neuronas que hacen “clic”: STDP y los circuitos spiking	5
3.4	La compuerta de saliencia: prestarle más atención a lo sorprendente	6
3.5	El genotipo: una receta con seis ingredientes	7
3.5.1	Cómo se cocina un recuerdo, paso a paso	8
3.6	La ley del umbral: la idea central del proyecto	9
3.6.1	Un detalle importante: no siempre se puede juntar	9
3.6.2	¿Cómo sabe un robot cuántas cosas distintas hay dando vueltas?	9
3.7	Las dos pruebas que tiene que pasar una memoria	10
4	Por qué se cuida tanto cómo está hecho el código	10
5	Los experimentos, contados uno por uno	11
5.1	exp01: probar las 576 recetas, todas	11
5.1.1	Cómo se mide “cuánto importa” cada decisión	11
5.2	exp02: la ley del umbral, puesta a prueba con números	12
5.3	exp03: chequear que ninguna decisión sea un botón desconectado	12
5.4	exp04: comparar formas distintas de construir la memoria por dentro	12
5.5	exp05: ¿esto funciona con fotos de verdad, no sólo inventadas?	12
5.6	exp06: probar todo esto con un robot que se mueve de verdad	13
6	Los cuatro resultados, en un minuto	14

7	La mejor receta encontrada, en detalle	14
7.1	Por qué hacen falta las dos decisiones juntas	14
7.2	Por qué es una buena idea para un robot de verdad, no sólo un truco de laboratorio	14
8	Cosas que todavía no funcionan del todo, dichas con franqueza	15
8.1	El circuito de neuronas de impulsos, y por qué no es un fracaso	15
8.2	El buffer viejo no es “malo”, es de otro objetivo	15
8.3	La pregunta más difícil: ¿y si no hay premio para adivinar?	15
9	Preguntas raras que puede hacer un profesor puntilloso	15
10	Banco de preguntas de examen, con respuestas modelo	16
11	Diccionario de palabras raras	17

Esta guía intenta explicar todo desde cero, como si nunca hubieras escuchado hablar del tema. Primero va la idea con una comparación de la vida real, después la explicación técnica, y después el código y los números para quien quiera ir más profundo.

1 La idea, contada como un cuento

Imaginate que tenés una mochila con espacio para diez cuadernos, nada más. Vivís yendo a la escuela todos los días y cada día pasan cosas: algunas aburridas y repetidas (la misma clase de todos los lunes), y alguna vez pasa algo raro e importante (un día ganaste un premio). La mochila se llena. ¿Qué cuaderno tirás cuando no entra uno nuevo?

La forma más simple de resolverlo es: tirá siempre el más viejo. No importa si era importante o aburrido, se va porque es el que hace más tiempo que está ahí. Eso es exactamente lo que hace hoy un robot construido con la arquitectura e-MDB: cuando su memoria se llena, tira lo más viejo, sin mirar si era importante.

EMBER es un proyecto que prueba una idea distinta: ¿y si el robot pudiera elegir tirar lo menos importante, en vez de lo más viejo? Para probarlo, se armaron un montón de “formas de tener memoria” distintas (576, para ser exactos), se probaron todas, una por una, sin trampa ni atajos, y se vio cuál se acuerda mejor de lo importante cuando la mochila se llena. Y se descubrió algo interesante: cuál es la mejor forma de armar la memoria depende de cuántas cosas repetidas hay dando vueltas comparadas con el tamaño de la mochila. Ese es todo el paper, resumido.

2 El problema, un poco más en serio

2.1 LOLA: el desafío de fondo

LOLA es el nombre que le ponen los investigadores a un problema concreto: construir un robot que aprenda cosas toda su vida, sin que nadie le diga de antemano qué tiene que aprender ni cuándo termina de aprender. No es como una tarea de la escuela que empieza y termina: es más como crecer. El robot tiene que seguir aprendiendo mientras trabaja, en un lugar que no eligió y que va cambiando.

2.2 e-MDB: la casa donde vive este problema

e-MDB es el nombre de una arquitectura de robot — un plano de cómo organizar todas las piezas de un robot que piensa — pensada justo para atacar el problema de LOLA. Cada pieza del robot se llama **Cognitive Node**, y cada cosa que le pasa al robot (lo que veía antes, qué hizo, qué pasó después, si le fue bien o mal) se guarda como un paquetito llamado **Episode**.

La pieza que guarda esos paquetitos se llama **EpisodicBuffer**, y adentro es, literalmente, una fila. En programación existe algo que se llama **deque** (se lee “dek”), que es exactamente una fila de gente: entra por un lado, y cuando se llena, sale el que está más adelante — el que llegó primero. **EpisodicBuffer** es eso, ni más ni menos: agrega episodios por un lado y, cuando se llena, tira el más viejo. No mira si el episodio era importante. No busca por parecido. No junta dos episodios parecidos en uno solo. Por eso el paper lo describe como “una base de datos sin criterio”: guarda cosas, pero no piensa cuáles.

El código de verdad -- `src/ember/core/genotype.py`

```
1 FIFO_GENOTYPE = Genotype(  
2     read=NearestNeighbour(),  
3     write=Append(),  
4     strength=Constant(),  
5     decay=NoDecay(),  
6     evict=FIFO(),  
7     reinforce=0.0,  
8 )
```

Esto de arriba es, en el lenguaje de EMBER, la receta exacta del **EpisodicBuffer** de e-MDB. Vamos a entender qué significa cada palabra de esa receta más adelante — por ahora quedate con la idea de que el buffer real ni siquiera busca cosas parecidas, y acá se le regaló esa capacidad igual (`read=NearestNeighbour`), para ser bien justos con él en la comparación. Si aun regalándole eso queda mal parado, no hay trampa posible.

2.3 ¿Por qué no alcanza con copiar lo que ya se conoce?

Hay ideas de hace muchos años que resuelven, cada una por separado, algo de lo que le falta al buffer de e-MDB: una manera de guardar información repartida para que no se pierda tan fácil (se llama **Sparse Distributed Memory**, la vemos en la próxima sección), una manera de reforzar lo que se repite (como un caminito de tierra que se marca más cuanto más se camina), una manera de prestarle más atención a lo sorprendente, y una manera de elegir qué olvidar según qué tan importante fue.

El problema es que casi siempre esas ideas se prueban cada una sola, armadas a medida para que se luzcan, y comparadas contra un rival fácil de ganar. Eso hace difícil saber si de verdad la idea es buena, o si ganó porque la compararon mal. EMBER prueba todas las ideas juntas, con las mismas reglas del juego para todas, incluido el buffer real de e-MDB.

3 Los conceptos, uno por uno

3.1 Sparse Distributed Memory (SDM): guardar un secreto repartido

Imaginate que tenés un secreto importante y en vez de escribirlo en un solo papel (que se puede perder o romper), lo escribís en veinticinco papelitos distintos y los escondés en veinticinco lugares

de tu casa que ya elegiste de antemano. Si alguien te pregunta el secreto, no vas a buscar un papel exacto: vas a mirar en varios de esos veinticinco lugares y con lo que encontrás ahí, aunque estén un poco arrugados o mojados, podés reconstruir el secreto completo. Eso es Sparse Distributed Memory (SDM), inventada por Kanerva en 1988.

Los “veinticinco lugares de tu casa” se llaman *hard locations*, y en EMBER hay 512 de esos lugares, elegidos al azar una sola vez al principio y que ya no cambian más. Cuando llega un recuerdo nuevo, EMBER no elige veinticinco al azar cada vez: elige siempre los mismos 25 lugares de esos 512, los que “más se parecen” al recuerdo (el 5 % más parecido).

El código de verdad -- `src/ember/memories/sdm.py`

```
1 def _conjunto_activo(self, key):
2     sims = self.H @ key
3     return np.argpartition(-sims, self.k_active - 1)[: self.k_active]
4
5 def write(self, key, value, pred_error=0.5):
6     k = unit(key)
7     s = self.strength_policy.initial(pred_error, self.store.novelty(k))
8     activas = self._conjunto_activo(k)
9     self.V[activas] += np.float32(s) * k
10    self.store.append(k, value, strength=s, contribution=s)
11
12 def _desalojar(self, idx):
13     clave = self.store.keys[idx].copy()
14     aporte = float(self.store.contribution[idx])
15     activas = self._conjunto_activo(clave)
16     self.V[activas] -= aporte * clave
17     self.store.remove(idx)
```

Con números

Con 512 lugares y activando el 5 % de ellos, cada recuerdo usa $512 \times 0.05 = 25$ lugares. Cuando se guarda un recuerdo con fuerza 1.8, esos 25 lugares suman 1.8 veces la clave del recuerdo. Si después ese recuerdo se borra, hay que restar exactamente lo mismo que se sumó: 1.8 veces esa misma clave, en esos mismos 25 lugares. `_conjunto_activo` siempre elige los mismos 25 lugares para la misma clave, así que restar es fácil de hacer bien.

Lo lindo de guardar así es que si el secreto llega un poco cambiado (con “ruido”), igual se puede reconstruir, porque no depende de un solo lugar perfecto: depende de la suma de varios lugares parecidos. Eso se llama degradación suave, y es justo lo que un robot necesita, porque sus sentidos nunca son perfectos.

3.2 El engrama: la huella que deja un recuerdo

Un engrama es la huella física que deja un recuerdo en el cerebro — como la marca que deja un pie en la arena. Cuanto más se repite algo, más marcada queda esa huella: eso se llama potenciación de largo plazo (LTP), y es la base de una frase famosa: “las neuronas que se activan juntas, se conectan entre sí”.

En EMBER, la versión chiquita de esa idea se llama Engram Neural Network (ENN): un recuerdo nuevo puede o crear una huella nueva, o — si ya hay una huella muy parecida guardada — sumarse

a esa huella y hacerla más fuerte, en vez de crear una copia.

El código de verdad -- `src/ember/memories/enn.py`

```
1 def write(self, key, value, pred_error=0.5):
2     k = unit(key)
3     store = self.store
4     store.strength *= np.float32(self.decay)
5
6     sims = store.similarities(k)
7     boost = self.strength_policy.initial(pred_error, store.novelty(k)
8     )
9
10    if sims.size and sims.max() >= self.merge_threshold:
11        i = int(sims.argmax())
12        store.keys[i] = unit(store.keys[i] + np.float32(self.eta *
13            boost) * k)
14        store.strength[i] += np.float32(boost)
15        return
16
17    store.append(k, value, strength=boost, contribution=boost)
```

Con números

Pensalo así: cada vez que ves algo parecido a lo que ya tenías guardado, esa huella se hace un poquito más fuerte, sumando un poco de fuerza cada vez (`store.strength[i] += boost`). Si ves la misma cosa aburrida treinta veces, sumando cerquita de 1.0 de fuerza cada vez, esa huella termina con una fuerza de más o menos 20. Un recuerdo raro que sólo pasó una vez, aunque haya sido muy sorprendente, arranca con una fuerza de 5 como mucho. 20 es mucho más que 5: la huella de lo aburrido y repetido termina pesando más que el recuerdo raro. Por eso lo aburrido a veces le gana el lugar a lo importante, si el mecanismo de “juntar recuerdos parecidos” está activado.

Ojo con esto

Esta es la trampa que hay que conocer: juntar recuerdos parecidos es útil para no llenar la memoria de cosas repetidas, pero si se usa junto con “sumar fuerza cada vez”, lo repetido termina ganándole a lo importante. Por eso la mejor receta que encontré EMBER (la vemos en la Sección 7) nunca junta recuerdos parecidos, aunque juntar sea, en otras condiciones, muy útil.

3.3 Neuronas que hacen “clic”: STDP y los circuitos spiking

Las neuronas de verdad no mandan un número continuo, mandan pulsos cortos, como un “clic”. Si la neurona A hace clic justo antes que la neurona B, la conexión entre ellas se hace más fuerte (porque parece que A causó que B se prenda). Si es al revés, la conexión se debilita. Esa regla se llama STDP, y solita agrupa neuronas en equipos que aprenden a prenderse juntos — un equipo así es, otra vez, un engrama, pero visto neurona por neurona.

El código de verdad -- src/ember/memories/spiking.py -- números por defecto

```
1 n_neurons=128, sparsity=0.15
2 a_plus=0.02, a_minus=0.022
3 tau_m=20.0, v_th=1.0, t_ref=2
4 n_presentations=30
5 cue_frac=0.5
```

Con 128 neuronas y `sparsity=0.15`, cada recuerdo prende cerca de 19 neuronas nada más — un equipo chico dentro de una red grande, para que los equipos no se confundan entre sí. Y hace falta ver el mismo patrón treinta veces (`n_presentations`) para que el equipo quede bien formado, porque `a_minus` es un poquito más grande que `a_plus`: la regla está pensada para que solamente lo que se repite de verdad deje huella, y el ruido aislado no alcance a marcar nada.

El límite de esto: funciona muy bien con patrones de “prendido/apagado” bien definidos, pero un robot ve el mundo con números continuos (una cámara, por ejemplo), no con patrones de prendido/apagado limpios. Un numerito distinto activa un equipo de neuronas un poco distinto, y ahí el truco se rompe un poco. Arreglar ese puente es trabajo para más adelante (lo que en el proyecto se llama PoC-5).

3.4 La compuerta de saliencia: prestarle más atención a lo sorprendente

Cuando te pasa algo sorprendente, tu cerebro le pone “más tinta” a ese recuerdo, para que te acuerdes mejor. Esa señal en el cerebro la manda la dopamina, y — esto es importante — no manda esa señal por cualquier sorpresa: la manda sobre todo cuando algo sale mejor o peor de lo que vos esperabas en términos de recompensa (un premio, una ganancia), no solo por ver algo raro.

El código de verdad -- src/ember/core/policies.py

```
1 class Constant:
2     def initial(self, pred_error, novelty):
3         return 1.0
4
5 class NoveltyGated:
6     gain: float = 2.0
7     def initial(self, pred_error, novelty):
8         return 1.0 + self.gain * novelty
9
10 class PredErrorGated:
11     gain: float = 2.0
12     def initial(self, pred_error, novelty):
13         return 1.0 + self.gain * pred_error
14
15 class BothGated:
16     gain: float = 2.0
17     def initial(self, pred_error, novelty):
18         return 1.0 + self.gain * novelty + self.gain * pred_error
```

Con números

Imaginate un recuerdo bastante distinto de todo lo que ya viste (“novedad” = 0.8 sobre 1) y medianamente sorprendente (“error de predicción” = 0.3 sobre 1). Cuatro formas de calcular su fuerza inicial: sin compuerta, siempre 1.0; solo por lo distinto que es, $1 + 2(0.8) = 2.6$; solo por lo sorprendente que es, $1 + 2(0.3) = 1.6$; por las dos cosas juntas, $1 + 2(0.8) + 2(0.3) = 3.2$. Fijate que la última no promedia, suma: si algo es raro Y sorprendente a la vez, se lleva el refuerzo de las dos señales, no solo de la más grande.

Ahora viene una parte contraintuitiva. Si medís cuánto cambia el resultado general al prender o apagar esta compuerta sobre las 576 formas de armar la memoria, el cambio parece chiquito: apenas 1.4%. Uno podría pensar “entonces la sorpresa no sirve para nada”. Pero eso pasa porque de las cuatro formas de elegir qué olvidar, tres de ellas ni siquiera miran la fuerza del recuerdo — calculan la sorpresa y la tiran a la basura sin usarla. Si mirás sólo las formas que sí usan la fuerza para decidir qué olvidar, ahí la sorpresa multiplica por más de catorce (14.4 veces) cuánto se acuerda de lo importante. La lección: si la señal de sorpresa no le llega a nadie que la escuche, no importa cuán buena sea la señal.

3.5 El genotipo: una receta con seis ingredientes

Pensá en cada forma de armar una memoria como una receta de cocina con seis decisiones que hay que tomar sí o sí. EMBER le pone nombre a cada una de esas seis decisiones y las llama “ejes”.

El código de verdad -- src/ember/core/genotype.py

```
1 class Genotype:
2     read: ReadPolicy
3     write: WritePolicy
4     strength: StrengthPolicy
5     decay: DecayPolicy
6     evict: EvictPolicy
7     reinforce: float
```

Decisión	Opciones	En criollo
read	nn / topk3 / radius	¿Buscás un solo recuerdo parecido, o varios a la vez?
write	append / merge	¿Cada cosa nueva es un recuerdo nuevo, o se puede juntar con uno parecido?
strength	constant / novelty / pred_error / both	¿Le prestás más atención a lo sorprendente, o a todo por igual?
decay	1.0 / 0.995 / 0.98	¿Los recuerdos se van debilitando solos con el tiempo?
evict	fifo / min_strength / min_utility / random	Cuando no entra algo nuevo, ¿qué tirás?
reinforce	0 / 0.5	¿Recordar algo lo hace más fuerte?

Si contás cuántas combinaciones hay probando todas las opciones de todas las decisiones, da $3 \times 2 \times 4 \times 3 \times 4 \times 2 = 576$. Y como son pocas y contables, se pueden probar TODAS, una por

una, sin adivinar ni aproximar. Nadie tiene que confiar en que un algoritmo “buscador” encontró la mejor: se prueban las 576 y se mira cuál ganó, de verdad.

Ojo con esto

Cuando arrancó este proyecto, alguien contó mal y pensaba que había 576 recetas distintas, pero en realidad dos de las seis decisiones no cambiaban nada en absoluto — estaban ahí escritas pero no hacían nada, como un botón que no está conectado a ningún cable. Sólo había 96 recetas realmente distintas. Por eso ahora existe una prueba automática (`exp03_axis_liveness`) que se corre siempre y avisa fuerte si algún botón vuelve a quedar desconectado.

3.5.1 Cómo se cocina un recuerdo, paso a paso

Toda la receta se junta en una sola función, que es la que de verdad decide qué pasa cada vez que llega un recuerdo nuevo:

El código de verdad -- `src/ember/core/memory.py`

```
1 def write(self, key, value, pred_error=0.5):
2     g = self.genotype
3     store = self.store
4     k = unit(key)
5
6     store.tick()
7     g.decay.step(store.strength)
8     novelty = store.novelty(k)
9     s = g.strength.initial(pred_error=pred_error, novelty=novelty)
10
11     objetivo = g.write.route(store, k)
12     if objetivo is not None:
13         store.strength[objetivo] += np.float32(s)
14         return
15
16     store.append(k, value, strength=s)
17     self._evict_until_fits()
```

Con números

Una memoria que sólo entran 3 recuerdos, con la receta ganadora (`append + both + sin debilitarse + tirar el más débil`). Llegan cuatro recuerdos distintos entre sí, con sorpresas `[0.1, 0.9, 0.2, 0.1]`:

recuerdo	fuerza inicial	qué queda guardado
1	3.2	[3.2]
2 (el sorprendente)	4.8	[3.2, 4.8]
3	3.4	[3.2, 4.8, 3.4]
4	3.2	no entra → se tira el más débil (el 1, con 3.2)

El recuerdo sorprendente (fuerza 4.8) nunca corrió peligro: siempre era el más fuerte. Si en cambio la fuerza fuera siempre 1.0 para todos (receta del buffer viejo), el “más débil” sería un empate entre todos, y en un empate se tira el que entró primero — que es, ni más ni menos,

tirar siempre lo más viejo.

3.6 La ley del umbral: la idea central del proyecto

Volvamos a la mochila de diez cuadernos. Si en tu semana pasan siempre las mismas cinco cosas una y otra vez, te conviene juntar los cuadernos repetidos en uno solo bien resumido: te sobra espacio para lo raro. Pero si en tu semana pasan veinte cosas distintas y ninguna se repite, juntar no te sirve de nada — no hay nada para juntar — y lo único que podés hacer es elegir bien qué tirar.

Esa idea, en números, se escribe así:

$$r = \frac{K_{\text{proto}}}{C}$$

K_{proto} es cuántas cosas distintas y repetidas hay dando vueltas, y C es cuántos cuadernos entran en la mochila. Si r es menor que 1 (menos cosas repetidas que espacio), conviene juntar. Si r es mayor que 1 (más cosas repetidas que espacio), juntar no ayuda y lo que importa es elegir bien qué tirar.

Ojo con esto

Para probar esta idea bien, en el experimento hay que tener cuidado de una cosa: si dejás que las cosas repetidas aparezcan más o menos veces según cuántas hay, te confundís y no sabés si lo que cambió fue r o fue otra cosa. Por eso, al probar distintos valores de r , se mantiene fijo cuántas veces aparece cada cosa repetida. Así, si algo cambia, sabemos que fue r y no otra cosa escondida.

3.6.1 Un detalle importante: no siempre se puede juntar

K_{proto} dice cuántas cosas distintas HAY. Pero para juntar dos recuerdos en uno hace falta que se parezcan bastante (más de 0.85 de parecido). Si en tu mundo las cosas “repetidas” en realidad se ven siempre un poco distintas entre sí (como dos fotos del mismo perro pero desde ángulos diferentes), juntar puede no dispararse casi nunca, y entonces aunque K_{proto} sea chico, terminás usando muchas más ranuras de las que pensabas. A eso se lo llama K_{efectivo} (cuántas ranuras se usan de verdad).

Mundo	Cuántas cosas distintas hay	Cuántas ranuras se usan	r de verdad	¿Importa juntar?
ordenado	10	10	0.50	Sí, 81.8 %
desordenado	10	35	1.75	Casi nada, 0.4 %

Y cuando se probó esto con fotos de verdad (el dataset CIFAR-100, con cien tipos de cosas distintas), pasó algo todavía más extremo: las fotos de una misma cosa se parecían tan poco entre sí (0.401 de parecido en promedio, contra el 0.85 que hace falta) que juntar prácticamente *nunca* se disparaba. No es que la regla de r estuviera mal: es que antes de preguntarte “¿en qué mundo estoy, ordenado o desordenado?”, primero hay que preguntar “¿acá siquiera se puede juntar algo?”.

3.6.2 ¿Cómo sabe un robot cuántas cosas distintas hay dando vueltas?

En el laboratorio esto se sabe de antemano porque se arma a propósito. Un robot de verdad no tiene esa suerte: sólo ve una fila larga de recuerdos y tiene que adivinar cuántos grupos distintos hay ahí adentro. Para eso se agrupan los recuerdos en montoncitos de prueba (2, 3, 4, ... montoncitos)

y se elige la cantidad de montoncitos que deja los grupos más “prolijos” — cada cosa bien cerca de las de su propio grupo y bien lejos de las de otro grupo.

El código de verdad -- `src/ember/data/prototypes.py`

```
1 SILUETA_MINIMA = 0.15
2
3 def estimate_n_prototypes(keys, k_max=64, n_boot=12, seed=0):
4     X = unit_rows(keys)
5     k_hat, puntajes = _mejor_k(X, ks, rng)
6     return PrototypeEstimate(k_hat=k_hat, scores=puntajes, ...)
```

Con números

Si le das puro ruido (nada que se parezca a un grupo de verdad), este método igual te va a devolver un número — por ejemplo “7 grupos” — pero con una prolijidad bajísima, tipo 0.06. Como 0.06 está por debajo de 0.15, EMBER sabe que ese “7” no significa nada y no lo usa. Es como cuando ves figuras en las nubes: siempre podés encontrar *alguna* forma, pero eso no quiere decir que las nubes tengan de verdad esa forma.

3.7 Las dos pruebas que tiene que pasar una memoria

Primero, sin apuro (bautizada R1 a R4): ¿la memoria puede acordarse de algo aunque se lo preguntes de forma incompleta o con ruido? Es como preguntarte “¿cómo se llamaba esa canción...” tarareando sólo un pedacito. Si no pasa esta prueba, ni vale la pena seguir mirando qué tan bien elige qué olvidar.

Después, bajo presión (bautizada T1 a T3, cuando llegan más recuerdos de los que entran): ¿se acuerda de lo raro e importante (T1)? ¿sigue funcionando bien aunque esté muy llena y algo apretada (T2)? ¿lo que aprendiste hace rato sobrevive a todo lo nuevo que siguió pasando (T3)?

4 Por qué se cuida tanto cómo está hecho el código

Estas reglas no son un capricho de prolijidad: cada una arregla un error de verdad que en algún momento hizo que un resultado publicado antes fuera incorrecto.

1. El núcleo de EMBER (las partes que algún día andarían adentro de un robot real) sólo usa una librería de matemática básica (numpy). No usa librerías pesadas de gráficos ni de entrenamiento de redes grandes, porque esas no tienen por qué andar dentro de una computadora chiquita montada en un robot.
2. Todo lo que tiene algo de azar (por ejemplo, sortear los 512 “lugares de la casa” de la SDM) parte siempre de un número semilla que se elige a propósito. Antes se usaba, por error, la dirección de memoria de una variable como semilla — algo que cambia solo, sin que nadie lo pida, entre una corrida y otra — y eso hacía que dos corridas del mismo experimento dieran resultados distintos sin ningún motivo real.
3. Cada mecanismo (juntar, olvidar, la compuerta de sorpresa, etc.) está escrito una sola vez, en un solo lugar. Antes había dos copias parecidas pero no idénticas, y comparar resultados entre ellas era comparar peras con manzanas sin que nadie se diera cuenta.

4. Cuando se guarda algo repartido en varios lugares (como en la SDM), y después se borra, hay que restar exactamente lo mismo que se sumó. Antes se restaba un poquito distinto de lo que se había sumado, y con el tiempo eso ensuciaba los números de a poquito, como una cuenta de banco que nunca cierra bien.
5. Ninguna de las seis decisiones de la receta puede ser un botón desconectado (ver el recuadro de la Sección 3.5). Hay una prueba automática que se asegura de esto todo el tiempo.
6. Cuando dos recetas empatan en el puntaje, se dice “empataron”, no se elige la más conveniente de las dos y se la muestra como si fuera la única ganadora.
7. Ningún número que aparece en el paper se escribe a mano: sale siempre de una corrida guardada, con su semilla y su configuración anotadas, así cualquiera puede volver a correrla y chequear que da lo mismo.

5 Los experimentos, contados uno por uno

Cada experimento es un archivo que se puede correr, y guarda sus resultados en una carpeta aparte para que nadie tenga que confiar de palabra en un número.

5.1 exp01: probar las 576 recetas, todas

Prueba las 576 combinaciones posibles sobre todas las pruebas (R1 a T3), y mira cuál ganó y en qué lugar quedó la receta del buffer viejo.

Lo que encontró: la receta ganadora es `append + both + sin debilitarse + tirar el más débil`, con puntaje 0.818 (sobre un máximo de 1.0). El buffer viejo saca 0.106, y queda entre los puestos 416 y 469 de 576 — ni el peor de todos (hay 107 recetas peores), pero sí bastante mal ubicado.

5.1.1 Cómo se mide “cuánto importa” cada decisión

El código de verdad -- `src/ember/nas/stats.py`

```

1 def eta_squared(records, axis, metric="mean"):
2     y = _valores(records, metric)
3     ss_total = ((y - y.mean()) ** 2).sum()
4     etiquetas = _etiquetas(records, axis)
5     ss_entre = 0.0
6     for opcion in np.unique(etiquetas):
7         grupo = y[etiquetas == opcion]
8         ss_entre += len(grupo) * (grupo.mean() - y.mean()) ** 2
9     return ss_entre / ss_total

```

Con números

Imaginate cuatro recetas de juguete, dos con `append` que sacan 0.9 de puntaje cada una, y dos con `merge` que sacan 0.3 cada una. El promedio de las cuatro es 0.6. Las de `append` están todas a 0.3 del promedio, las de `merge` también están a 0.3 del promedio — y como no hay ninguna otra diferencia entre ellas, toda la diferencia de puntaje se explica completamente por esa sola decisión. En este ejemplo de juguete, esa decisión explicaría el 100 % de la diferencia.

5.2 exp02: la ley del umbral, puesta a prueba con números

Prueba 24 situaciones distintas de r , con 5 repeticiones cada una, sobre las 576 recetas, en tres tamaños de mochila distintos (10, 20 y 40).

Momento	Valor de r	¿Cuánto importa juntar?	¿Cuánto importa elegir bien qué tirar?
Mundo ordenado	$r \leq 0.5$	73.0 %	8.2 %
En el medio	$0.5 < r < 1$	52.4 %	10.6 %
Mundo desordenado	$r \geq 1$	1.6 %	16.3 %

El cambio de un mundo a otro pasa siempre cerca de $r \approx 0.88$, sin importar si la mochila era chica, mediana o grande. Eso es justo lo que hacía falta demostrar: no depende del tamaño de la mochila en sí, depende de la relación entre lo que hay y lo que entra.

5.3 exp03: chequear que ninguna decisión sea un botón desconectado

Ya lo contamos en el recuadro de la Sección 3.5: agrupa recetas “hermanas” que sólo difieren en una decisión, y si en algún grupo cambiar esa decisión no cambia nunca el resultado, avisa. Encontró que las seis decisiones sí hacen algo (antes, dos de ellas no hacían nada), así que el espacio de 576 recetas de verdad es de 576 recetas distintas.

5.4 exp04: comparar formas distintas de construir la memoria por dentro

Acá no se compara la receta (las seis decisiones), sino la “anatomía” completa: SDM, ENN, un circuito de neuronas de impulsos, un mix de SDM con ENN, y el buffer viejo.

Anatomía	R1	R2	R3	R4	Promedio	¿Pasa?	T1	T2	T3
SDM	0.672	0.788	1.000	0.978	0.859	sí	1.000	0.365	0.875
ENN	0.691	0.938	1.000	1.000	0.907	sí	0.750	0.344	0.375
Neuronas de impulsos	0.190	0.146	1.000	0.499	0.459	no	—	—	—
Mix SDM+ENN	0.507	0.433	0.950	0.588	0.619	sí	0.675	0.158	0.719
Buffer viejo	0.691	0.938	1.000	1.000	0.907	sí	0.038	0.254	0.000

Sólo el circuito de neuronas de impulsos no pasa la primera prueba (la de “acordarse aunque sea con pistas incompletas”), y no es porque esté mal construido — en pruebas anteriores, sobre patrones limpios de prendido/apagado, funcionaba perfecto — sino porque acá tiene que trabajar con números continuos, que es justo el tipo de entrada que le cuesta.

El dato más interesante es SDM contra ENN bajo presión: la SDM se acuerda perfecto de lo raro ($T1 = 1.000$), pero ENN se acuerda peor ($T1 = 0.750$) usando exactamente la misma compuerta de sorpresa y la misma forma de elegir qué olvidar. La diferencia es la que vimos en el ejemplo de la Sección 3.2: ENN junta recuerdos parecidos y eso le suma fuerza extra a lo repetido, empujando a lo raro hacia la salida.

5.5 exp05: ¿esto funciona con fotos de verdad, no sólo inventadas?

Repite **exp02** pero usando fotos reales de cien tipos de cosas (CIFAR-100), en vez de vectores inventados en la computadora.

Lo que se encontró: con este tipo de foto, juntar recuerdos parecidos casi nunca se dispara (0.401 de parecido contra 0.85 necesario), así que la decisión de juntar o no juntar deja de importar casi nada (entre 0.0 % y 0.4 % según la prueba), sin importar cuánto valga r . No es que la ley esté mal: es que hace falta chequear primero si juntar es siquiera posible en ese mundo, antes de preguntarse en qué régimen de r estás.

5.6 exp06: probar todo esto con un robot que se mueve de verdad

Los experimentos anteriores usan fotos ya sacadas de antemano. Acá se usa MiniGrid, un jueguito donde un personaje se mueve por un laberinto y tiene que acordarse de algo que vio al principio para elegir bien al final — un robot recibe las cosas que le pasan por sus propias decisiones, no una lista ya armada.

El código de verdad -- src/ember/envs/minigrid.py

```
1 class OneStepPredictor:
2     def surprise(self, obs, action, next_obs):
3         pred = self.W @ self._entrada(obs, action)
4         err = next_obs - pred
5         self.W += self.lr * np.outer(err, self._entrada(obs, action))
6         return np.clip(np.linalg.norm(err) / (2.0 * self._escala + 1e
7             -8), 0.0, 1.0)
8
9 class RewardPredictionError:
10     def surprise(self, obs, action, reward):
11         pred = self.w @ self._entrada(obs, action)
12         err = reward - pred
13         self.w += self.lr * err * self._entrada(obs, action)
14         return np.clip(abs(err) / (2.0 * self._escala + 1e-8), 0.0,
15             1.0)
```

La diferencia entre estas dos clases es sólo qué intentan adivinar antes de que pase: una adivina qué va a ver, la otra adivina si va a ganar algo. Esa única diferencia cambió todo el resultado:

1. Con el personaje moviéndose al azar y adivinando “qué voy a ver”, la memoria buena no se acuerda de nada especial (0.0 %): lo raro visualmente y lo importante de verdad se parecen demasiado.
2. Se probó hacer que el personaje se mueva más (avance en vez de girar), para tener cuatro veces más momentos importantes. No cambió nada: más datos con la misma señal mala siguen siendo una señal mala.
3. Se cambió qué intenta adivinar el predictor: en vez de “qué voy a ver”, ahora adivina “voy a ganar algo”. Ahí sí: la memoria buena pasó de acordarse 0.0 % a acordarse 52.6 % de las veces importantes, mientras que el buffer viejo se quedó igual de mal (2.6 %), porque el buffer viejo ni siquiera mira esa señal.

La conclusión: prestarle más atención a lo sorprendente funciona de verdad, en un mundo real y no inventado — pero sólo si lo que llamamos “sorpresa” está bien elegido. Sorprenderse por ver algo raro no alcanza; hay que sorprenderse por ganar o perder algo.

6 Los cuatro resultados, en un minuto

1. Qué mecanismo funciona mejor depende del cociente $r = K_{\text{proto}}/C$: con poco r , juntar recuerdos explica el 73 % de la diferencia entre recetas; con mucho r , sólo el 1.6 %, y ahí manda elegir bien qué olvidar.
2. La mejor receta se acuerda de 0.983 de lo importante contra 0.050 del buffer viejo, cambiando sólo dos de las seis decisiones.
3. El circuito de neuronas de impulsos no pasa la primera prueba, porque fue pensado para otro tipo de entrada.
4. Con datos reales aparecen dos condiciones extra que hacen falta para que todo esto funcione: que las cosas repetidas se parezcan de verdad entre sí, y que la sorpresa esté anclada a ganar o perder algo, no sólo a ver algo raro.

7 La mejor receta encontrada, en detalle

{read: nn, write: append, strength: both, decay: 1.0, evict: min_strength, reinforce: 0.0}

En criollo: cada cosa nueva se guarda aparte (nunca se junta con otra), con una fuerza que depende de cuán rara y sorprendente fue. Esa fuerza no se debilita nunca sola, ni tampoco cambia cuando se la recuerda: queda fija desde el momento en que se guardó. Cuando la memoria se llena, se tira siempre la más débil — que, por construcción, es la menos importante que había guardada.

7.1 Por qué hacen falta las dos decisiones juntas

Esta receta cambia sólo dos cosas respecto del buffer viejo: le agrega la compuerta de sorpresa, y cambia “tirar lo más viejo” por “tirar lo más débil”. Si le cambiás sólo una de las dos, no pasa nada:

Configuración	Promedio	T1	T2	T3
Buffer viejo	0.106	0.050	0.269	0.000
+ sólo la compuerta de sorpresa	0.106	0.050	0.269	0.000
+ sólo tirar lo más débil	0.106	0.050	0.269	0.000
+ las dos juntas	0.818	0.983	0.469	1.000

Es lógico si lo pensás: si todos los recuerdos pesan lo mismo (sin compuerta de sorpresa), “tirar el más débil” es lo mismo que “tirar el primero que entró” — son la misma cosa disfrazada. Y si nadie mira la fuerza al elegir qué tirar, da igual lo bien que la compuerta de sorpresa calcule esa fuerza: nadie la usa.

7.2 Por qué es una buena idea para un robot de verdad, no sólo un truco de laboratorio

Esta receta nunca junta recuerdos — así que nunca puede pasarle lo que le pasaba a ENN, donde lo repetido termina pesando más que lo importante. Y como nunca depende de si estás en un mundo ordenado o desordenado, un robot que la use no necesita saber en cuál de los dos está: le funciona igual en ambos casos.

8 Cosas que todavía no funcionan del todo, dichas con franqueza

8.1 El circuito de neuronas de impulsos, y por qué no es un fracaso

Que no pase la primera prueba no significa que STDP esté mal: en pruebas anteriores, sobre patrones bien definidos, armó equipos de neuronas perfectos. Lo que falta es un puente entre “números continuos, como los de una cámara” y “patrones bien definidos, como los que STDP entiende bien”. Ese puente es trabajo para más adelante.

8.2 El buffer viejo no es “malo”, es de otro objetivo

Compararlo así de generoso (regalándole una búsqueda por parecido que no tiene de verdad) y que aun así quede mal ubicado no es una crítica personal a quien lo diseñó: e-MDB fue pensado resolviendo otros problemas primero. Es simplemente una manera de mostrar, con números, que ese componente en particular no alcanza para el problema de LOLA.

8.3 La pregunta más difícil: ¿y si no hay premio para adivinar?

Ojo con esto

Este es el punto más flojo del trabajo, y conviene decirlo así de claro si alguien pregunta. La solución que funcionó (adivinar si vas a ganar algo) necesita que *haya* algo que ganar. Pero LOLA, el problema de fondo, es justamente sobre un robot que puede pasar mucho tiempo sin ningún premio de por medio. En esos tramos, la única señal que queda es “qué tan raro se ve”, y esa señal es justo la que no funcionó. Hay ideas candidatas para esos casos (por ejemplo, que el robot se dé sus propios premios por explorar cosas nuevas), pero ninguna se probó todavía acá: queda anotado como trabajo pendiente, no escondido.

Otras cosas que faltan probar: sólo se usó un tipo de cámara/extractor de fotos, un solo juego de fotos, y un solo tamaño de memoria; sólo se probó en un laberinto de juguete con dos formas de moverse; y nada de esto se probó todavía en un robot de verdad, con sensores que fallan y un mundo que cambia solo con el tiempo.

9 Preguntas raras que puede hacer un profesor puntilloso

- **Radius** (una de las formas de buscar recuerdos) no usa un número fijo de “cuán parecido tiene que ser”: lo compara siempre contra el mejor resultado de esa consulta en particular. Si usara un número fijo, en un espacio de 32 dimensiones casi todo es bastante poco parecido a todo, y el número fijo terminaría siempre encontrando un solo recuerdo — ni más ni menos que buscar el más parecido nada más, disfrazado de otra cosa.
- Para comparar “cuánto importa cada decisión” se usa una cuenta (**variance_share**) que reparte el 100 % entre todas las decisiones, en vez de una cuenta que se satura en 100 % para casi cualquier decisión no nula — porque en un espacio sin ruido de fondo (como este, donde correr la misma receta da siempre el mismo resultado), esa segunda cuenta deja de servir para comparar.
- Cuando dos o más recetas empatan en un puesto del ranking, EMBER siempre lo cuenta como un rango de puestos (por ejemplo “entre el puesto 416 y el 469”), nunca como un solo número — porque elegir el extremo más lindo de un empate de decenas de recetas sería mentir un poquito con los números.

- El error de predicción de `exp06` lo calcula un modelito simple, nunca lo escribe una persona a mano — porque esa señal es justo la que decide todo el resultado, y ponerla a mano sería, en los hechos, decidir el resultado de antemano.

10 Banco de preguntas de examen, con respuestas modelo

P: ¿Por qué el efecto de la compuerta de sorpresa parece tan chico (1.4 %) si es un mecanismo tan citado?

R: Porque de las cuatro formas de elegir qué olvidar, sólo una escucha la fuerza de los recuerdos. En las otras tres, la sorpresa se calcula y se tira. Si mirás sólo la forma que sí la escucha, la sorpresa multiplica por más de catorce cuánto se acuerda de lo importante. El número chico no dice que el mecanismo sea débil: dice que hay que mirar con más cuidado antes de sacar una conclusión.

P: ¿Por qué separar “receta” de “anatomía”?

R: Porque responden preguntas distintas. Las 576 recetas prueban, con la misma anatomía, cuál es la mejor manera de decidir qué guardar y qué tirar. Las cuatro anatomías (SDM, ENN, neuronas de impulsos, mix) prueban si esa forma de guardar recuerdos puede siquiera reconstruir algo desde una pista incompleta. Mezclar las dos preguntas confunde: por ejemplo, el buffer viejo es el peor entre anatomías, pero no es el peor entre las 576 recetas (hay 107 peores).

P: ¿Por qué mantener fijo cuántas veces se repite cada cosa al comparar distintos valores de r ?

R: Porque si no lo fijás, subir la cantidad de cosas distintas también baja automáticamente cuántas veces se repite cada una, y no sabrías si lo que cambió el resultado fue r o esa repetición escondida. Fijarlo aísla a r como la única cosa que cambia.

P: ¿Por qué se le regala al buffer viejo una búsqueda por parecido que no tiene de verdad?

R: Para que nadie pueda decir después que se lo comparó de forma injusta o empobrecida a propósito. Si aun regalándole esa ventaja queda mal ubicado, la conclusión es más fuerte, no más débil.

P: Si la ley del umbral no funciona igual con fotos reales, ¿eso arruina el resultado principal?

R: No, porque en realidad hay que chequear dos cosas por separado: primero, si juntar recuerdos parecidos es siquiera posible en ese mundo (mirando cuánto se parecen las cosas repetidas entre sí); y sólo si eso es posible, recién ahí importa cuántas ranuras se usan de verdad. Con las fotos de CIFAR-100 la respuesta a la primera pregunta ya es que no, así que ni llega a hacer falta la segunda.

P: ¿Qué pasa con la sorpresa si el robot no tiene ningún premio por el que guiarse?

R: Es la limitación más sincera del trabajo. Lo que funcionó (adivinar si vas a ganar algo) necesita que haya algo para ganar. Sin eso, sólo queda la sorpresa por “ver algo raro”, que es justo la que no funcionó bien. Hay ideas para esos casos (premiarse a uno mismo por explorar), pero ninguna se probó todavía.

P: ¿Por qué la mejor receta nunca junta recuerdos, si juntar es lo que más importa en un mundo ordenado?

R: Porque juntar acumula fuerza cada vez que se repite algo, y con suficientes repeticiones esa fuerza le termina ganando a un recuerdo raro guardado una sola vez, por sorprendente que haya sido. La mejor receta evita ese problema de raíz: como nunca junta, nunca puede acumular fuerza de esa manera, y por eso funciona igual de bien esté el mundo ordenado o desordenado.

P: Los números cambiaron una vez (de 35 veces mejor a 14.4 veces mejor). ¿Por qué confiar igual en ellos?

R: Porque cambiaron justo cuando se arreglaron errores de verdad (semillas que no se podían repetir, cuentas que quedaban mal después de borrar recuerdos, dos copias del mismo mecanismo con reglas levemente distintas). Y lo que no cambió es lo más importante: la receta ganadora es exactamente la misma de antes, y la forma general de la ley del umbral también se mantuvo. Un número que sobrevive a que le arreglen los errores de abajo es más confiable que uno que nunca se puso a prueba.

11 Diccionario de palabras raras

Palabra	Qué quiere decir
SDM	Guardar un recuerdo repartido en muchos lugares en vez de uno solo, para no perderlo tan fácil.
ENN	La versión chiquita de EMBER de un engrama: recuerdos que se pueden juntar y hacerse más fuertes.
STDP	La regla de las neuronas: si una se prende justo antes que la otra, se conectan más fuerte.
LIF	El modelo de neurona que se usa acá: va juntando carga hasta un límite y ahí “dispara”.
LTP	Que una conexión entre neuronas se haga más fuerte con el uso repetido.
e-MDB	La arquitectura de robot que tiene el problema que EMBER intenta arreglar.
LOLA	El problema de fondo: un robot que aprende toda su vida, sin que nadie le diga qué ni cuándo.
PoC	Una prueba chica para validar una idea antes de construir algo más grande.
NAS	Buscar automáticamente cuál es la mejor forma de armar algo, entre muchas posibles.
R1–R4	Las pruebas de “¿te acordás bien?”, sin apuro de espacio.
T1–T3	Las pruebas de “¿te acordás bien?”, con la memoria ya llena.
r	Cuántas cosas distintas hay, dividido cuánto entra en la memoria.
C	Cuánto entra en la memoria.
K_{proto}	Cuántas cosas distintas y repetidas hay dando vueltas.
K_{efectivo}	Cuántas ranuras se usan de verdad después de intentar juntar lo repetido.

Guía armada a partir de `paper/main.tex`, `docs/ESTADO.md`, `CLAUDE.md`, la skill `ember-domain`, y el código de `src/ember` del repositorio EMBER.